

Strukturalni paterni

1. **Facade patern** se uvodi kako bi se pojednostavila komunikacija sa dijelom sistema za pametno planiranje boravka. Umjesto da controller ili korisnički dio aplikacije direktno koristi više klasa kao što su SmartPlanner, Preferencija, Event, Atrakcija i UgostiteljskiObjekat, uvodi se klasa PlaniranjeFacade koja nudi jednostavan interfejs za kreiranje, prikaz i ažuriranje plana boravka. Na taj način se smanjuje složenost korištenja sistema i sakriva unutrašnja logika planiranja.

2. **Adapter patern** se uvodi zbog potrebe povezivanja sistema sa vanjskim email servisom za slanje newslettera. Klasa Newsletter ne treba direktno zavisiti od konkretnog vanjskog servisa, jer se taj servis u budućnosti može promijeniti. Zbog toga se uvodi interfejs IEmailServis i klasa EmailServisAdapter koja prilagođava metode vanjskog servisa potrebama GoSarajevo sistema. Na taj način se omogućava lakša zamjena email servisa bez izmjene osnovne klase Newsletter.

3. **Observer patern** se uvodi radi upravljanja pretplatama i obavještavanjem korisnika. Klasa RegistrovaniKorisnik može se pretplatiti na određene kategorije sadržaja (eventi, nove atrakcije itd), a kada dođe do promjene u sistemu (npr. admin doda novi event), svi pretplaćeni korisnici automatski dobijaju obavijest. Uvode se interfejs IObserver sa metodom Obavijesti(), interfejs ISubject sa metodama Pretplati(), OtkaziPretplatu() i ObavijestSvePretplatnike(), te konkretne implementacije KorisnikObserver i npr. EventSubject. Na taj način se izbjegava čvrsta veza između klasa koje generišu promjene i onih koje na njih reaguju.

4. **Strategy patern** se uvodi u funkcionalnosti pametnog planiranja boravka. SmartPlanner klasa treba generisati prijedlog plana boravka po različitim strategijama ovisno o preferencijama korisnika. Umjesto grananja unutar jedne klase, uvodi se interfejs IStrategijaPlaniranja sa metodom GenerišiPlan(Preferencija preferencija), a svaka vrsta planiranja implementira svoju klasu (HistorijskaStrategija, GastronomskaStrategija, KombiniranaStrategija itd.). SmartPlanner tada delegira generisanje plana odabranoj strategiji, što olakšava dodavanje novih vrsta preporuka bez mijenjanja osnovne logike planera.

5. **Singleton patern** se uvodi za upravljanje globalnim podacima sistema. Klasa ApplicationDbContext treba biti dostupna kroz cijelu aplikaciju kao jedna jedina instanca, kako se ne bi kreiralo više konekcija na bazu podataka ili učitalo konfiguracijske parametre više

puta. Uvođenjem Singleton paterna osigurava se da postoji tačno jedna instanca ove klase u cijelom životnom vijeku aplikacije.

6. **Decorator patern** se uvodi radi dinamičkog proširivanja prikaza i opisa atrakcija i ugostiteljskih objekata. Osnovna klasa Atrakcija ili UgostiteljskiObjekat daje bazični prikaz podataka (naziv, adresa, opis), ali se na taj prikaz može dinamički nadograđivati: recenzije, oznaka "skriveni dragulj", badge popularnosti, ili informacije o radno vremenu. Umjesto nasljeđivanja za svaku kombinaciju, uvodi se interfejs ILokacijaPrikaz i konkretni dekoratori poput RecenzijeDekorator, SkriveniDraguljDekorator i PopularnostDekorator.

7. **Template Method patern** se uvodi za kreiranje newsletter sadržaja. Operater za korisničku komunikaciju kreira različite vrste newslettera (sedmični pregled evenata, sezonske preporuke, hitne obavijesti), a sve dijele isti generalni proces tj. priprema zaglavlja, unos tijela poruke, dodavanje footer-a i slanje. Apstraktna klasa NewsletterSablon definira metodu KreirajNewsletter() kao šablon koji poziva redoslijed koraka, dok su specifični koraci (npr. FormirajTijelo()) apstraktni i prepisuju se u konkretnim klasama SedmičniNewsletter, SezonskiNewsletter i slično.